

(If something is messed up, or not written correctly, feel free to fix it xoxo)

QUIZ

The dependency structure of the packages of a release component should represent

a:

Question 1
Incorrect
Mark 0.00 out of 10.00
Flag question

The dependency structure of the packages of a release component should represent a

Select one:

- ☐ a. Directed Cyclic Graph
- ☒ b. Tree ✖
- ☐ c. Binary tree
- ☒ d. Directed Acyclic Graph
- ☐ e. List

Your answer is incorrect.

Which of the following are good design principles

Question 2
Correct
Mark 10.00 out of 10.00
Flag question

Which of the following are good design principles

Select one or more:

- ☐ a. High coupling
- ☒ b. Low coupling ✔
- ☒ c. High cohesion ✔
- ☐ d. Low cohesion
- ☐ e. None of the above

Your answer is correct.

Abstract packages

Question 3
Incorrect
Mark 0.00 out of 10.00
Flag question

Abstract packages

Select one or more:

- ☒ a. Should be depended upon
- ☐ b. Should contain abstract classes
- ☒ c. Doesn't matter if they are stable or not ✖
- ☐ d. Should depend on other packages
Binary tree
- ☒ e. Should be stable

Your answer is incorrect.

Which of the following are good criteria for grouping classes together into packages?

Question 4
Partially correct
Mark 6.67 out of 10.00
Flag question

Which of the following are good criteria for grouping classes together into packages?

Select one or more:

- ☒ a. All classes in a package should be reused together ✔
- ☐ b. All classes in a package should be in an inheritance relation
- ☐ c. All classes in a package should implement the same interfaces
- ☒ d. All classes in a package should be reusable
- ☒ e. All classes in a package should change for the same reason(s) ✔

The stability of a package in a design Answer: D, E

Question 5
Incorrect
Mark 0.00 out of 10.00
Flag question

The stability of a package in a design

Select one or more:

- ☐ a. Should be as low as possible
- ☐ b. Does not depend on the other packages
- ☒ c. Should to be as high as possible ✖
- ☐ d. Depends both on the incoming and outgoing dependencies
- ☐ e. Is not that important as long as the package depends on more stable ones

Your answer is incorrect.

Which of the following are good practices for layered architecture

Question 1
Partially correct
Mark 3.00 out of 10.00
Flag question

Which of the following are good practices for a layered architecture

Select one or more:

- ☒ a. Changes made in one layer of the architecture should not impact components in other layers ✔
- ☐ b. A layer may depend on all the lower layers
- ☒ c. A layer may depend only on the immediate lower layer ✖
- ☒ d. A layer should depend on interfaces implemented by the lower layers
- ☐ e. None of the above

Which of the following statements is true? (MVC, MVVM, ViewModel) + All options are good

Question 2
Partially correct
Mark 4.00 out of 10.00
Flag question

Which if the following statements is true?

Select one or more:

- ☐ a. Several Views can have references to the same ViewModel
- ☒ b. MVVM is a variant of the MVC pattern ✔
- ☒ c. The ViewModel represents a Model for the View ✔
- ☒ d. Views can be designed independent from the code behind in MVVM
- ☐ e. The ViewModel is a Façade to the Model

Suitable solution(s) for decoupled, distributed architecture of heterogeneous, independent components/service is(are):

Question 3
Incorrect
Mark 0.00 out of 10.00
Flag question

Suitable solution(s) for a decoupled, distributed architecture of heterogeneous, independent components/services is(are):

Select one or more:

- ☒ a. Broker
- ☒ b. MVC ✖
- ☐ c. Client-Server
- ☒ d. 3-tiered ✖
- ☒ e. Peer-to-peer

The broker component in the broker pattern:

Question 4
Partially correct
Mark 3.33 out of 10.00
Flag question

The Broker component in the Broker pattern:

Select one or more:

- ☒ a. Holds a collection of Servers ✔
- ☐ b. Represents a single point of failure
- ☒ c. Returns the most suitable server based on a client request
- ☐ d. Holds a collection of Clients
- ☐ e. Holds both a collection of Clients and a collection of Servers

Which statements are true about MVC:

Question 5
Incorrect
Mark 0.00 out of 10.00
Flag question

Which statements are true about MVC:

Select one or more:

- ☒ a. The View and the Controller depend on the Model
- ☐ b. Cannot be applied in a layered architecture
- ☒ c. It is an instance of the Observer DP
- ☒ d. The Model depends on the View and the Controller ✖
- ☒ e. It is highly decoupled ✖

Aggregates in the context of DDD:

Question 1
Correct
Mark 10.00 out of 10.00
Flag question

Aggregates in the context of DDD

Select one or more:

- ☒ a. Group together objects that are a unit related to data change ✔
- ☐ b. May have several roots
- ☐ c. The aggregated objects are also accessible from outside
- ☒ d. The root entity has global identity while the inner entities have local identity ✔
- ☐ e. The aggregated objects can exist even if the root is deleted

REST services are different from SOAP-based services because:

Question 2
Correct
Mark 10.00 out of 10.00
Flag question

REST services are different from SOAP-based services because:

Select one or more:

- ☐ a. REST does offer built-in security and transaction management
- ☒ b. REST is a set of architectural principles ✔
- ☒ c. A response to a REST request can be in several formats (i.e. XML, HTML, plain text, JSON) ✔
- ☒ d. REST services are stateless ✔
- ☐ e. REST reads cannot be cached

What is true about SOAP based services:

Question 3
Correct
Mark 10.00 out of 10.00
Flag question

What is true about SOAP based services:

Select one:

- ☐ a. They are based on standards such as XML
- ☐ b. The interface offered by the service is described by using the Web Services Description Language
- ☐ c. The architecture is an instance of the Broker pattern
- ☐ d. SOAP is a protocol
- ☒ e. All of the above ✔

Considering Service design principles we can state that:

Question 4

Correct

Mark 10.00 out of 10.00

Flag question

Considering Service design principles we can state that

Select one or more:

- ☐ a. Services should maintain state information
- ☒ b. Services should be discoverable no matter where they actually reside ✓
- ☐ c. We should know the underlying business logic in order to use services
- ☒ d. Services are loosely coupled ✓
- ☒ e. We should be able to reuse services in various business flows ✓

In the context of domain driven design the following statements hold:

Question 5

Correct

Mark 10.00 out of 10.00

Flag question

In the context of Domain driven design the following statements hold:

Select one or more:

- ☐ a. If the Value Objects are shared, they should be mutable
- ☒ b. Value Objects are not tracked while Entities should be trackable ✓
- ☒ c. Entities represent uniquely identifiable concepts ✓
- ☒ d. Value Objects represent attributes without identity ✓
- ☐ e. Value Objects cannot contain other Value Objects

Some best practices in the iDesign method are:

Some best practices in the iDesign method are:

Select one or more:

- ☐ a. Engines may receive queued calls
- ☒ b. Clients do not call Engines ✓
- ☐ c. Engines may call each other
- ☐ d. Clients do publish events
- ☒ e. Managers do not queue calls to more than one Manager in the same use-case ✓

In the iDesign method:

In the iDesign method:

Select one or more:

- ☒ a. Engine services encapsulate business rules and activities ✓
- ☐ b. A Manager service encapsulates a set of unrelated use-cases
- ☐ c. There is a 1-to-1 relationship between Manager and Engine services
- ☐ d. The iDesign Architecture is a strict Layered one
- ☒ e. A Manager service addresses the sequence volatility ✓

Very small services (microservices) have consequences such as

Very small services (microservices) have consequences such as

Select one or more:

- ☒ a. Decrease performance if many services need to cooperate ✓
- ☐ b. Enable better performance
- ☐ c. Are not appropriate for independent database updates
- ☒ d. Are usually self-sufficient ✓
- ☒ e. Are not appropriate for transactional database updates ✓

The following is true about orchestration and choreography:

The following is true about orchestration and choreography

Select one or more:

- ☒ a. Orchestration involves a coordinating service that handles the microservices interaction ✓
- ☒ b. The orchestrator is a single point of failure in a process ✓
- ☒ c. Choreography is a decentralized, event-driven approach ✓
- ☒ d. Choreography allows for removing or adding services anytime from the event stream ✓
- ☐ e. None of the above

The following is true in the iDesign method:

Select one or more:

- ☐ a. It is the responsibility of the Engine component to translate business transactions into CRUD operations
- ☒ b. It is the responsibility of the ResourceAccess component to translate business transactions into CRUD operations ✓
- ☒ c. A ResourceAccess service encapsulates the volatility in accessing a Resource ✓
- ☐ d. A Resource can be just a database
- ☐ e. Utility Services are called only by Manager components

The responsibilities of the Data access components are

The responsibilities of the Data access components are

Select one or more:

- ☐ a. Map the data structure to the domain model structure
- ☐ b. Contain some business logic
- ☒ c. Connect to the database ✓
- ☒ d. Store the raw data ✓
- ☒ e. Execute queries ✓

Considering how inheritance is reflected in a relational database structure the following is true:

Considering how inheritance is reflected in a relational database structure the following is true

Select one or more:

- ☒ a. A table can contain the objects of a concrete class in the inheritance tree ✓
- ☐ b. Inheritance can be applied to tables the same way as it is applied to classes
- ☒ c. A table can contain the objects of all the classes in an inheritance tree ✓
- ☒ d. A table can contain the objects of any class in an inheritance tree ✓
- ☐ e. There is no way to reflect inheritance in a relational database structure

Active record violates the following principle(s):

Active Record violates the following principle(s):

Select one or more:

- ☐ a. Open-closed
- ☐ b. Interface segregation
- ☒ c. Single responsibility ✓
- ☐ d. Dependency inversion
- ☐ e. Liskov Substitution

The following is true about the Data Mapper:

The following is true about the Data Mapper:

Select one or more:

- ☒ a. Can be used with Table Data Gateway ✓
- ☒ b. Its responsibilities are to retrieve data from the storage and to map it on the business logic structures ✓
- ☒ c. Adds a layer that decouples the Business logic from the Data Access ✓
- ☐ d. Should be used with Transaction Script
- ☐ e. Its responsibilities are to map the Business logic structures on the presentation

Transaction script is an example of business logic design based on:

Transaction script is an example of business logic design based on

Select one or more:

- ☐ a. Domain driven decomposition
- ☒ b. Functional driven decomposition ✓
- ☐ c. Volatility driven decomposition
- ☒ d. Use-case driven decomposition ✓
- ☒ e. Transaction driven decomposition ✓

A page controller

A Page Controller

Select one or more:

- ☒ a. Forwards the data to be displayed to the appropriate view ✓
- ☐ b. Uses Command objects to delegate actions
- ☒ c. Invokes model objects to process the data ✓
- ☒ d. Decodes the URL and extracts the data for action ✓
- ☐ e. Handles duplicated features

The transform view:

The Transform View

Select one or more:

- ☒ a. Enables portability ✓
- ☐ b. Embeds programming logic into a HTML page
- ☒ c. Acts like a mapper between the XML model and the HTML to be displayed ✓
- ☒ d. Makes it easy to change the appearance of a web page ✓
- ☐ e. Builds the structure of the HTML page based on a template

The Unit of Work pattern is used to:

The Unit of Work pattern is used to

Select one or more:

- ☐ a. Eliminate deadlocks
- ☒ b. Maintain the changes of all objects affected by a business transaction ✓
- ☒ c. Enhance the performance by avoiding several small database updates ✓
- ☐ d. Maintain all the transactions involved in a use-case
- ☒ e. Maintain the consistency of the database ✓

A transaction has the following properties:

- ☐ a. Interface segregation (i.e. the transaction needs to have a well defined interface)
- ☒ b. Atomicity (i.e. the sequence of operations cannot be split further into the individual operations) ✓
- ☒ c. Consistency (i.e. the system should be in a consistent state before and after the transaction) ✓
- ☐ d. Dependency Inversion (i.e. the transaction should not depend on other transactions)
- ☒ e. Isolation (i.e. the transaction cannot be interrupted) ✓

The following is true about concurrency control

The following is true about concurrency control

Select one or more:

- ☒ a. Pessimistic concurrency control prevents conflicts ✓
- ☐ b. Isolated or immutable data needs concurrency control
- ☐ c. Optimistic concurrency control relies on hope that conflicts will not happen
- ☐ d. Conflicts cannot be avoided
- ☒ e. Optimistic concurrency control detects conflicts ✓

A site displaying a blurred, low-resolution copy of the images instead of the high-resolution image is an example of using the following design pattern:

A site displaying a blurred, low-resolution copy of the images instead of the high-resolution image is an example of using the following design pattern:

Select one or more:

- ☐ a. Bridge
- ☐ b. Factory method
- ☐ c. Decorator
- ☒ d. Proxy ✓
- ☐ e. Adapter

What strategies is the Bridge Design pattern applying:

What strategies is the Bridge Design pattern applying:

Select one or more:

- ☒ a. Encapsulate the volatilities ✓
- ☒ b. Favor composition over inheritance ✓
- ☒ c. Depend on abstractions instead of specific classes ✓
- ☒ d. Be open for extension and closed for modification ✓
- ☒ e. Concern separation ✓

The following is true about the Composite pattern:

The following is true about the Composite pattern

Select one or more:

- ☒ a. Both the primitive elements (leaves) and the containers (composites) need to respect the same interface ✓
- ☐ b. The components should never have a reference to their container (composite)
- ☒ c. Composites may contain both leaves and composites ✓
- ☐ d. The container operations should always be placed in the Component interface
- ☐ e. Composites may contain only leaves

From the Clients perspective, the objective of the Adapter Design pattern is to

From the Clients perspective, the objective of the Adapter Design pattern is to

Select one:

- ☐ a. Decouple the Target from the Adaptee
- ☐ b. Encapsulate how objects used by the Client are created
- ☐ c. Provide a uniform interface that the Adaptees need to implement
- ☐ d. Override the behavior of the Adaptee
- ☒ e. Allow using objects that are not compliant with the Target interface ✓

If we want to define a standard car and several options that could be added to build a custom car we would use the following design pattern:

Select one or more:

- ☐ a. Proxy
- ☐ b. Bridge
- ☐ c. Factory method
- ☒ d. Decorator ✓
- ☐ e. Adapter

The following is an example of applying the Observer DP:

Select one or more:

- ☐ a. Data Access Objects
- ☐ b. Data Mappers
- ☐ c. Broker
- ☒ d. Listeners in Java ✓
- ☒ e. MVC ✓

Which DP(s) is/are more suitable to model the concept of a thread considering its lifecycle that includes the following possible states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED:

Select one or more:

- ☐ a. Composite
- ☐ b. Strategy
- ☒ c. State ✓
- ☐ d. Adapter
- ☐ e. Decorator

Which DP is/are more suitable to model the concept of a stock trader (agent de bursa) that gets buying or selling orders from a client:

Select one or more:

- ☒ a. Proxy ✓
- ☐ b. Composite
- ☒ c. Command ✓
- ☐ d. Bridge
- ☐ e. Strategy

Which DP is/are more suitable to model a transportation from the hotel to the airport considering the existing options: by train, bus or taxi:

Select one or more:

- ☐ a. Adapter
- ☐ b. Decorator
- ☐ c. Composite
- ☒ d. Strategy ✓
- ☐ e. State

Considering the Chain of responsibility DP which of the following is true?

Select one or more:

- ☐ a. It is similar to the Observer DP because the sender holds a collection of receivers
- ☒ b. It is different from the Command DP because the request is not modeled as an object ✓
- ☐ c. It is different from the Decorator DP because it involves sequence of linked objects
- ☐ d. It is similar to the Decorator DP because all the linked objects have the same interface
- ☒ e. It is different from the Command DP because it may have several request handlers ✓

The meaning of the Open-Closed principle is: Answer: A,D

The meaning of the Open-Closed Principle is:

Select one or more:

- ☐ a. A class should be inheritable
- ☐ b. A module should depend on abstractions that are instantiated depending on the specific context
- ☐ c. A functional change can be done by modifying the existing code
- ☒ d. A functional change should not affect the existing code ✓
- ☐ e. None of the above

The meaning of the Dependency Inversion principle includes:

The meaning of the Dependency Inversion Principle includes:

Select one or more:

- ☒ a. Specific/Concrete classes should depend on Abstractions ✓
- ☐ b. High level layers should depend on the implementations in the low level layers
- ☒ c. High level layers should depend on the interfaces that are implemented in the low level layers ✓
- ☐ d. Low level layers depend on the high level layers
- ☐ e. None of the above

The meaning of the Liskov Substitution Principle is:

The meaning of the Liskov Substitution Principle is:

Select one or more:

- ☒ a. An object of the superclass should always be replaceable by an object of the subclass ✓
- ☐ b. An object of the subclass should always be replaceable by an object of the superclass
- ☐ c. A subclass can do less than the superclass
- ☒ d. A subclass should do at least what the superclass does ✓
- ☐ e. A subclass can do something completely different than the superclass

The meaning of the Single responsibility Principle is:

The meaning of the Single Responsibility Principle is:

Select one or more:

- ☐ a. A class should have just one method
- ☐ b. A class should do just one thing
- ☒ c. A module should have just one reason to change ✓
- ☒ d. A module should be responsible to just one user/stakeholder/actor ✓
- ☐ e. All of the above

The meaning of the interface segregation is:

The meaning of the interface Segregation Principle is:

Select one or more:

- ☐ a. Classes should depend on as many interfaces as possible
- ☒ b. Classes should depend only on the interfaces they need ✓
- ☐ c. Interfaces should include as many features as possible
- ☒ d. Interfaces should include only cohesive features (i.e.needed together) ✓
- ☐ e. None of the above

GOOD TO KNOW STUFF

(if info too ez, skip. No shaming pls)

- Webserver = aplicatie care preia requesturi
- Webclient = browsers, mobile apps etc..
- httpS = (S = secure)cripteaza informatia
- Cookie = variabile in care putem salva de ex. numele user-ului, istoric-ul etc.

Tine minte istoricul, ca data viitoare sa ne recomande produse anume. Se salveaza pe calculator, si browser-ul se uita pe calculator sa vada daca avem un cookie pentru site-ul respectiv. Durata de viata in jur de 30 de zile

- Session = cat timp avem browserul deschis, datele sunt salvate pe server. Daca se inchide, stergem acele date. De ex. intr-un session user.id == null -> redirect to Login; else stay on page and show Greetings

- MVC (main ideas):

M > classes that represent the data of the app

V > displays model data

C > handle browser requests, call views to return responses (in the controller every method is a http end-point)

PROBLEME REZOLVATE

SEND MORE HELP PLS

1)

Consider the following code snippet. Is there any class design principle violated? If so, how would you fix this?

```
class Vehicle {  
    public String brand;  
    public double price;  
    public int productionYear;  
    public String toString(String formatType) {  
        switch(formatType) {  
            case "JSON":  
                // implement JSON formatting here  
                return jsonFormattedString;  
                break;  
            case "XML":  
                // implement XML formatting here  
                return xmlFormattedString;  
                break;  
            default:  
                // implementation of default formatting  
                return defaultFormattedString; } }  
}
```

The above code violates the Open/Closed principle, since adding a new format type would change the class, not extend it. A solution would be to create an interface and several classes that implement that interface, each doing the desired operation. This way creating a new type would only extend our code and not modify it.

```
Public interface FormatToString{  
    String toString();  
}  
  
Public JSONFormat implements FormatToString{  
    String JSONString  
    String toString(){  
        //JSON formatting  
    }  
}  
  
etc.
```

2) Chain of responsibility vs Observer

Question 3

Not yet answered

Marked out of 1.000

Flag question

Show the common features and the differences between the Chain of responsibility and Observer design patterns. Give brief examples of designs that include each pattern.

Both the chain of responsibility and the observer are **behavioral design patterns**, that means that it is concerned with the algorithms and the responsibilities of the objects in the pattern.

Observer	Chain of responsibility
update when something happened, and get the answer from the one who could solve it	send the request through the pipe, and maybe solve the problem
possible negative outcome is a bad sign	possible negative outcome is acceptable
single filter should be enough for the request sending to the possible receiver(s)	
Has two possible ways of sending the approach of the notification, push (from the 'sender') and pull (the ones which are waiting take the state)	one-directional, if we want to translate, push format, in which the handler pushes the request down the pipe and every node solve/forward
relation of one-to-many between object	sort of pipeline , linked list relation between objects

Example of observer pattern: - **example systems: delivery systems, or broadcast**

- all observers do the same in manner of execution (bicycle couriers as example) and there is a slight difference either in performance or in the possibility to give the job to one of the couriers, as example taking into consideration distance from the restaurant, or price of delivery
- all observers need to do the job **simultaneously**, which means the request is a 'broadcast', as example a message which needs to be interpreted by every observer, sent by a command post

Chain of responsibility pattern: - **example systems: complex request handling, imposed order request approval**

- there is a big pool of requests, and each request varies in a big fashion, maybe containing only two sub-requests from total 10, but another contains 4, a third contains only one, and so on. If every sub-request is considered that one handler can assure it, it is the easiest way to send the request down the pipe, and wait at the finish for a response, so that sub-requests were solved somewhere in the meantime
- different types of requests can be 'approved' only by different ranks in this responsibility chain, example of signing an official paper, goes from the secretary up to the CEO in a 'chain'

3) Chocolate bar SOLID

Question 2

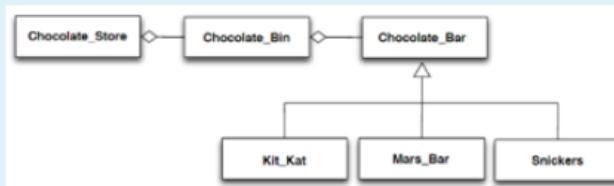
Not yet answered

Marked out of 1.000

Flag question

Consider the class diagram and the code sequence below. Analyze it with respect to the SOLID principles. Redesign the classes or the code if you identify a violation and explain why your design respects the principles.

```
class Chocolate_Bar {  
.....  
    public void restock() {  
        if (getAmount() <= LowStock)  
            if (barName == "Snickers") add(new Snickers());  
            if (barName == "Kit_Kat") add(new Kit_Kat());  
            if (barName == "Mars_Bar") add(new Mars_Bar());  
    }  
}
```



The above example violates the Open/Closed principle, since creating a new chocolate bar will always modify our current code, and the open closed principles states that it should be open for extension and closed for modification.

To solve this issue, the following solution could be used:

```
Public interface ChocolateBar{
```

```
    Public void restock();}
```

```
Public class Snickers implements ChocolateBar{
```

```
    Public void restock(){ if (getAmount() <= LowStock)
```

```
        Add(new Snickers());
```

4) Universal File Reader (UFR)

Question 1

Not yet answered

Marked out of 1.000

Flag question

You are to design a Universal File Reader. The UFR should be able to read, print, and convert to text any file. A configuration file is used to store the mappings between a file extension and the appropriate commands to read, print and convert to text. For example the configuration file would contain lines like the following:

```
.ps ghostview lpr pstotext
```

```
.gif xv printpic none ("none" means that there is no application that can perform the required operation)
```

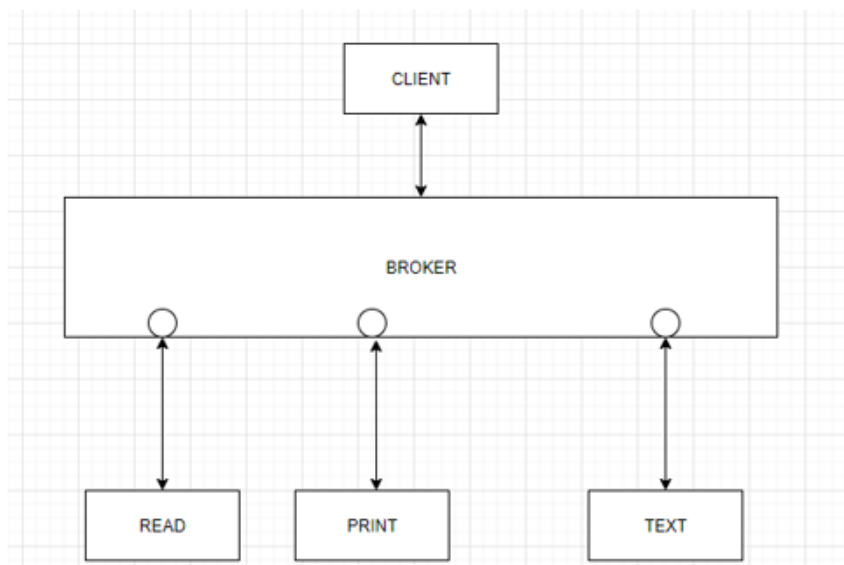
At the command line the user types: "ufr -t operation_type file_name"

operation_type: can be "read", "print", or "text"

file_name: is the name of the file to process. If a directory is given instead of a simple file name the whole directory should be processed.

Briefly analyze possible architectural alternatives and sketch an architecture for the system. Justify the proposed solution.

We could use the **BROKER** pattern. Where the client sends a command, and the Broker will decide which service will execute that command. operations are totally separated one from the other, and the broker could simply choose the best solution for the command line operation to be done



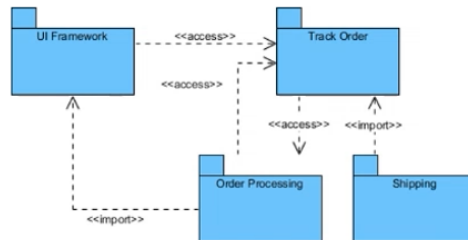
The parsing and the decision process is the Broker's 'problem' and it will connect to the needed service, based on the given command, to do for the client. The **configuration file** would be in help of the broker to decide and send the needed information with the command itself to the services, since it contains the necessary mappings between a file extension and the appropriate commands to read/print/convert text.

Parameters like *file_name* and *directory*, would be given as commands to the broker by the client and executed properly.

Alternative solutions: **layered** architecture, the method of iDesign, since each operator can have a manager, which could execute via engines, and these could connect to the necessary tools

5) Considering the Package Design

2. Considering the Package Design principles you know, how would you evaluate the following design? Justify. Provide a better design if the case.



Calculate instability and check if all packages depend on more stable ones:

Instability = incoming_dependencies / (incoming_dependencies + outgoing_dependencies)

UI Framework package: $1/(1+1)=1/2$; **Track order p.:** $2/3$; **Order Processing p.:** 0;

Shipping p.: 0

We have a cycle in dependencies. **NOT OK!!**

Formula: $\text{outgoing}/(\text{outgoing}+\text{incoming})$

UI: $1/(1+1) = 1/2$

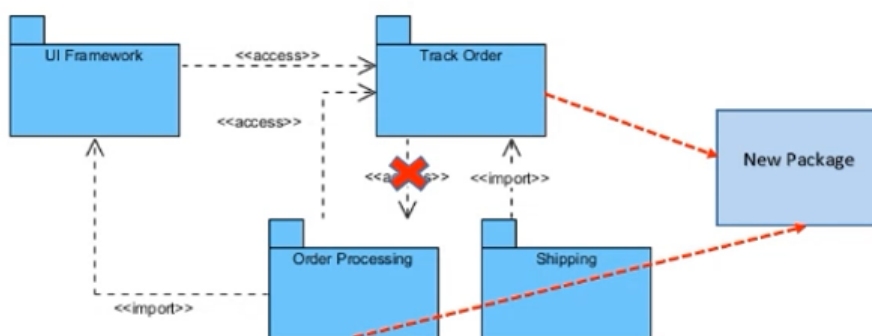
Track order: $1/(1+3) = 1/4$

Order processing: $2/(2+1) = 2/3$

Shipping: $1/(1+0) = 1$

Pachetul trebuie sa depinda de pachete care sunt mai stabile decat el

2.



sol from teacher

6) Event process SOLID

1. Discuss the following code snippet considering the SOLID principles. If the case, provide a better alternative.

```
if(event instanceof PreProcess) {  
    doSomething();  
} else if(event instanceof PostProcess) {  
    doSomethingElse();  
} else if(event instanceof Processing) {  
    doSomethingNew();  
}
```

Open closed principle is violated

Solutions from teacher:

```
void method(Event event){  
    event.doSomething();  
}
```

... or the Event is an attribute of the calling class => no need for parameter

```
interface Event {  
    void doSomething();  
}
```

```
class PreProcess implements Event {  
    @Override  
    public void doSomething() { // TODO }  
}
```

```
class PostProcess implements Event {  
    @Override  
    public void doSomething() { // TODO }  
}
```

```
class Processing implements Event {  
    @Override  
    public void doSomething() { // TODO }  
}
```

7) Consider an image recognition system with the following tasks: filtering, ...

3. Consider an image recognition system with the following tasks: filtering, feature extraction, labeling, clustering, scene reconstruction; assume that there is no a priori known fixed order in which these have to be called; also, the system should be easy to update with new algorithms.

i. Which architectural style is most appropriate for the given system?

ii. Give a sketch of sample architecture for the given system. Explain the interaction between the parts of the architecture.

iii. Discuss the reason for selecting a given style for the given system. Discuss possible disadvantages (if any) for the given choice.

i.

ii.

3. Pipe and Filter vs. Broker/Mediator or Manager (orchestration responsibility)

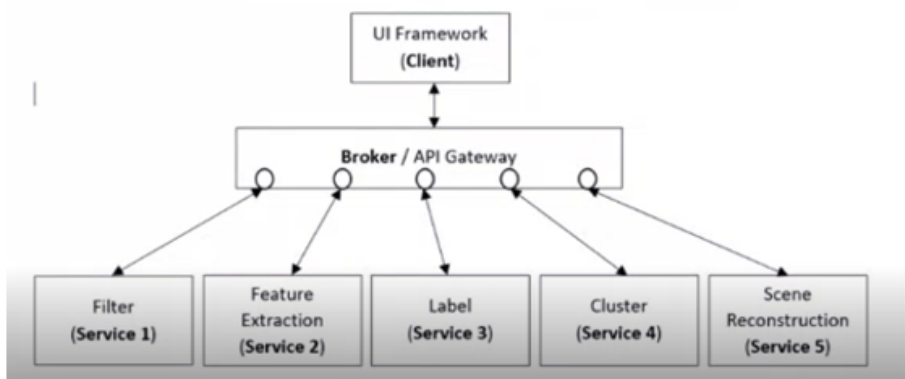


diagram from teacher

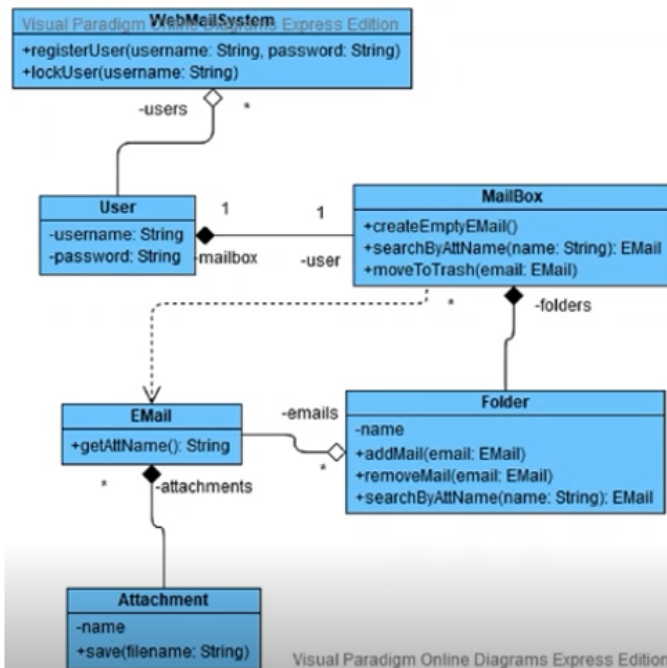
iii. Lower fault tolerance for server or broker failure;

8) Consider an e-mail system containing the classes WebMailSystem, EMail, User, Folder, Attachment and MailBox. Draw a conceptual UML class diagram for the classes and their semantic relationships and cardinalities. Regarding the following functionalities:

4. Consider an e-mail system containing the classes WebMailSystem, EMail, User, Folder, Attachment and MailBox. Draw a conceptual UML class diagram for the classes and their semantic relationships and cardinalities. Regarding the following functionalities:

- create a new, empty email
- move an email to the Trash system folder
- find emails that contain an attachment with a given name
- save an attachment to a file
- register a new user
- lock a user account

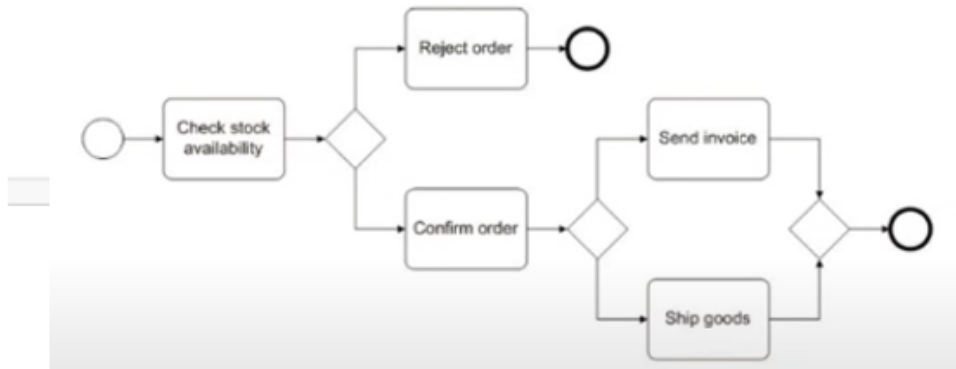
Justify textually based on the principles you know which of the classes is responsible for each functionality and add the methods to the classes.



Solution from teacher

9) Consider a simplified Order management process.

5. Consider a simplified Order management process as depicted. Identify possible services that could be defined and design the overall service architecture. Specify the type of the identified services.



Solution from teacher

10) Many sensors - if a problem is noticed

In an airplane, there are many sensors: speed, altitude, cabin pressure, fuel level, etc. The monitoring system performs different checks on the sensor data. If a problem is noticed, the system either shows a warning to the pilot (e.g. low on fuel), or in a dangerous situation may react automatically (e.g. by dropping oxygen masks).

Briefly analyze possible architectural alternatives and draw a diagram that describes your system architecture. Briefly explain in words how the system works (highlight the patterns you included). State the two most important advantages of using this architecture. State the two most important disadvantages of using this architecture.

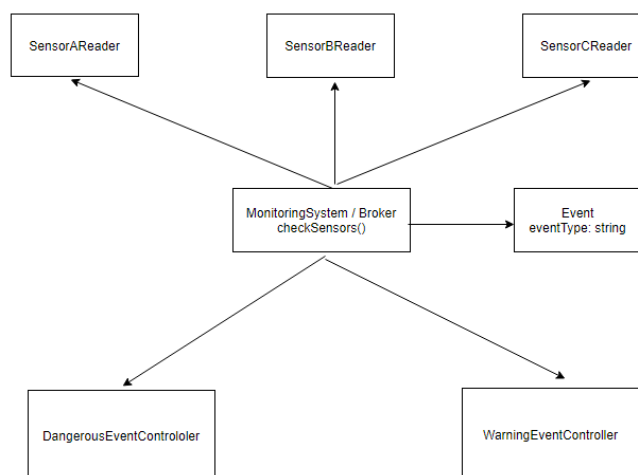
Consider a home security system having sensors for: movement, temperature, video camera etc. The monitoring system performs different checks on the sensor data. If a problem is noticed, the system sends a warning to the owner via SMS, or in a dangerous situation may react automatically (e.g. by starting the water sprinkler).
Briefly analyze possible architectural alternatives and draw a diagram that describes your system architecture. Briefly explain in words how the system works. State the two most important advantages of using this architecture. State the two most important disadvantages of using this architecture.

Event-driven architecture: The system only performs an action when a certain event (problem from sensor) is detected.

More exactly: Broker architecture (not Mediator because the actions are not chain reactions / there is a simple action for an event)

Advantages: Asynchronous (An event decides what happens) / Components can be easily changed, Simple implementation

Disadvantages: Not all events can be anticipated/ Reliability: Everything depends on the broker, always need to communicate with it



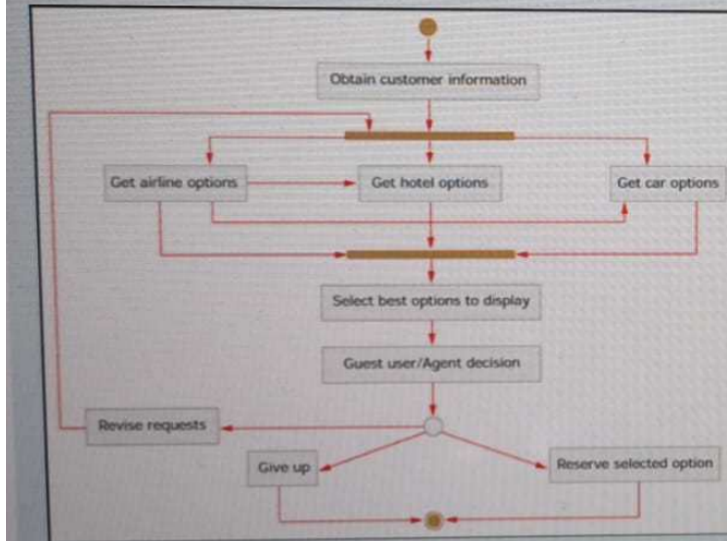
More info: <https://www.tiempodev.com/blog/event-driven-architecture/>

11) Web app for travel agency / vacation

Consider a web app for a travel agency that allows users to configure their vacation in terms of airplane reservations, hotel bookings, car rentals etc. The activity diagram is below. Considering that:

- different services have different providers (airlines, hotels, car rentals etc) and a service can be provided by several providers
- customers need complete travel packages that respect some global constraints (i.e. budget and time period).

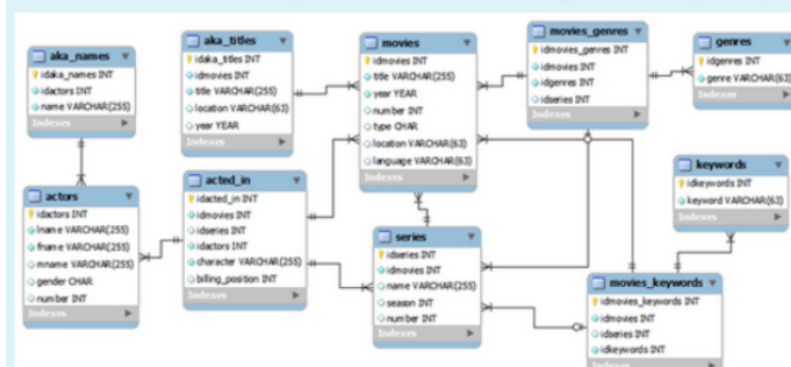
What architecture style would you employ? Draw a draft architecture highlighting the patterns you are using (i.e. who does the service composition and how?)



SOA architecture should fit perfectly to this one, because services are designed to be reusable, with low coupling, independent of each other and to abstract the logic behind. This architecture lets the services virtualize how the capability is performed, and where and by whom the resources are provided, enabling multiple providers and consumers to participate together in shared business activities.

12) Database schema

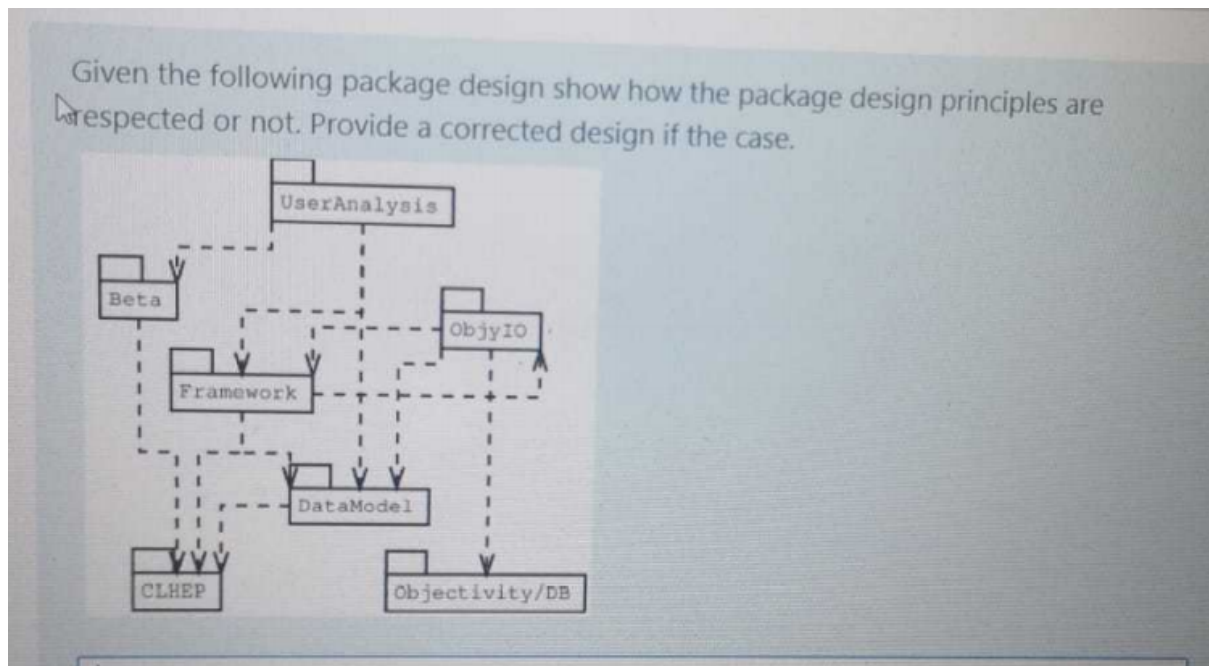
Consider the database schema below. You have to design the Business Logic layer that uses the DB in a movie management app. What are the architectural alternatives (with pros and cons)? Sketch a design of the BL layer and justify your choices.





?

13) Package design principles



Rezolvare :

Package Design Principles sunt **incalcate** ,deoarece:

ACYCLIC DEPENDENCIES PRINCIPLE (ADP) nu este respectat.Exista un ciclu intre pachetele Framework si ObjyIO.Pentru a corecta putem adauga un nou pachet intre cele doua .Atat Framework ,cat si ObjyIO vor fi dependente de noul pachet .

STABLE DEPENDENCY PRINCIPLE (SDP) : pentru a verifica,vom calcula pentru fiecare pachet instabilitatea($Instability\ I = Ce / (Ca + Ce)$,Ca – Afferent coupling (incoming dependencies) ,Ce – Efferent coupling (outgoing dependencies))

ps-examen-2020.pdf x cursuri PS.pdf x proiectPS.pdf x +

Fișier | C:/Users/eti12/Downloads/cursuri%20PS.pdf

130 din 785

Vizualizare pagină | Citii cu voce tare | Desenare | Evidențiere | Ștergere

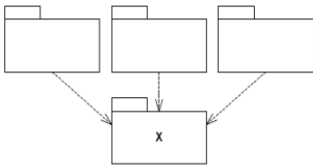
Ca – Afferent coupling (incoming dependencies)
 ■ How responsible am I?

Ce – Efferent coupling (outgoing dependencies)
 ■ How dependant am I?

Instability $I = Ce / (Ca + Ce)$

Example for X:

$Ca = 3, Ce = 0 \Rightarrow I = 0$ (very st



Tastați aici pentru a căuta

96% 22°C ENG 2:11 PM 2021-06-19

Pentru UserAnalysis $I = 1$

Pentru Beta $I = 0.5$

Pentru Framework $I = 3/4 = 0.75$,Pentru ObjyIO $I = \frac{3}{5} = 0.6$

Pentru DataModels $I = \frac{1}{3}$, Pentru CLHEP $I = 0 \Rightarrow$ foarte stabil, Objectivity $I = 0$

$\Rightarrow$ Se observa ca nu toate pachetele sunt stabile ($I = 0$)

14) In a web-based class management system, the instructor ...

Question 1

Not yet answered

Marked out of 1.000

Flag

In a web-based class management system, the instructor can see the list of classes he taught in the last 10 semesters, can select a class and see the students enrolled in it, and he can set the grade of each enrolled student if the gradebook is not closed. When a grade is changed the student gets a notification by e-mail.

Based on a layered architecture propose a solution covering all the 3 layers. Highlight any other patterns that are included in your design. Discuss at least 2 advantages and 2 disadvantages of your solution

Three layers are:

- **Data Access** layer $\Rightarrow$ the access to the database itself, with the students, classes for the teacher
- **Business** layer $\Rightarrow$ the input from the presentation layer converted, using logic and different needed services
- **Presentation** layer $\Rightarrow$ in short terms the UI of the system, the interaction of the users with the program (login etc)

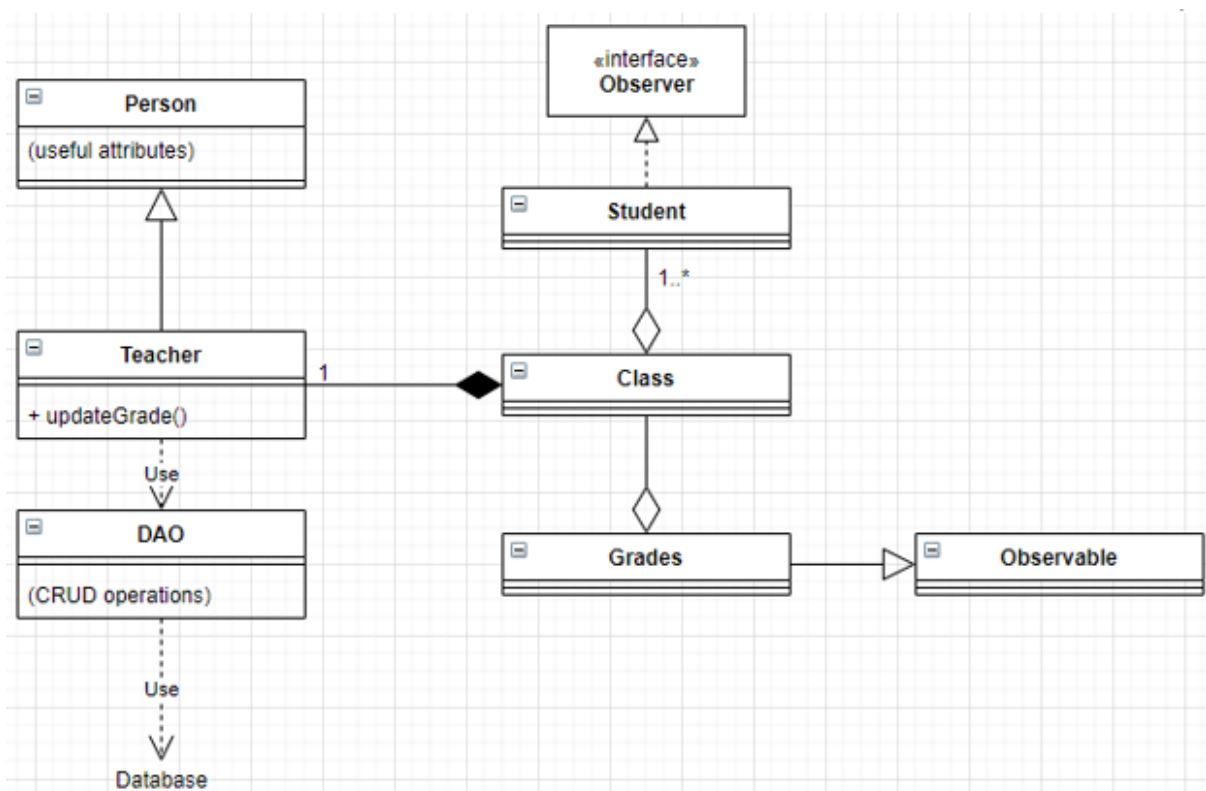
A suitable design pattern would be the observer pattern since the students need to be notified in case of a change of grade, which means they need to have a constantly available link between the modification system and their user(s). When a grade modifies, the student will receive an email (via email sending service).

Pro:

- The student gets notified if the grade is updated
- Loose coupling thanks to the observer pattern (new student can be easily added to the system)

Cons:

- The observer is limited since it is not an interface, if any other observer behavior is needed, there is need for a totally new Observer structure
- Observable protects a few methods => we need inheritance to make them work as we want



15) Security Central Office - CRUD

A Security Central Office (SCO) monitors the security systems of several clients. The security system of a client is of one of several possible types. There can be different types of contracts between the clients and the SCO. Depending on the contract there is a different insurance type attached to the contract. You have to design a management system for SCO. The system should allow the following operations: CRUD operations for clients, CRUD operations for the security system of a client, monitor a security system of a client, react to an event, CRUD operations for the contract between clients and SCO.

Name the architectural pattern(s) that you will use (not design pattern). Draw a diagram that describes your system architecture. Briefly explain the responsibilities of your components. State the two most important advantages of using this architecture. State the two most important disadvantages of using this architecture.

Layered architecture (because why not)?

Presentation Layer: The UI/ Views

Business Layer: Models (Contract, Client, Insurance, SecuritySystem) ClientService, SecuritySystemService, ContractService

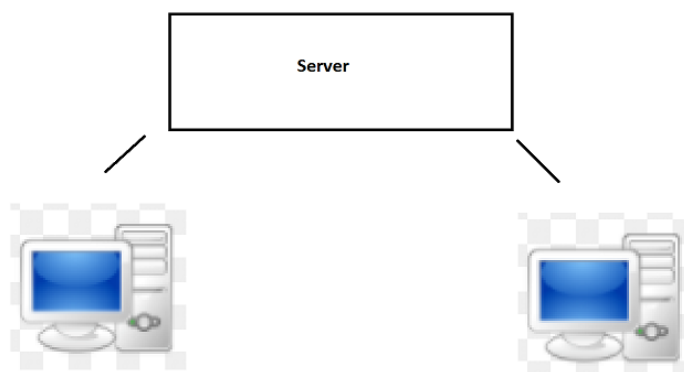
Data Access layer: ClientDAO, SecuritySystemDAO, ContractDAO

16) Remote Pair Programming

You have to design a system that allows Remote Pair Programming (i.e. two remote programmers work on the same code in real-time: the "driver" edits the code while the "observer" reviews the code and points to objects on the "drivers" desktop.) Roles can be swapped. The work should be saved and the system should ensure that both work on the same unique copy.

What architecture pattern/style would you apply to implement the system described? Draw a draft architecture highlighting the patterns you are using. Briefly justify your solution showing where the unique working version should be kept.

Client-server (because it is good for document sharing and stuff) + we need a central server in which we can store the project(since the exercise says that they have access to a COPY of the project) .



17)

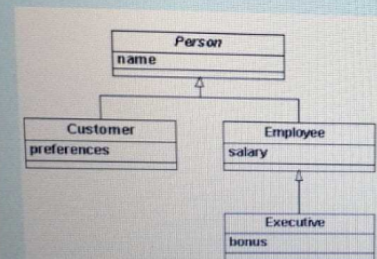
Consider Instagram-like app that allows you to take a picture, apply a digital filter to it, and then share it with other Instagram users or upload the picture to your favorite social networking service. The user may choose from several available filters and also from several social network services. Sketch a high-level architecture of the system. Analyze possible architectural alternatives and justify the proposed solution.

Client-server (motivatiia ii mai jos in doc la "Client-server")

Also, SOA, because a service could be defined for each action (filterService, uploadService, etc..)

18)

Consider the inheritance tree above. How would you model a relational database schema to store the instances of these classes? Briefly justify



Exista 3 metode de rezolvare :

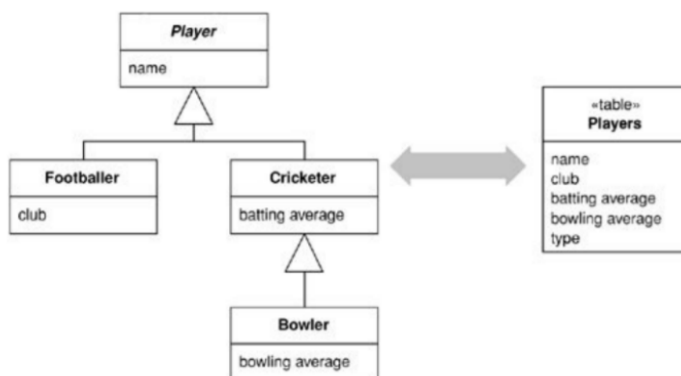
- Inheritance

Object-Relational Structural Patterns

- Single Table Inheritance
- Class Table Inheritance
- Concrete Table Inheritance

Metoda 1:

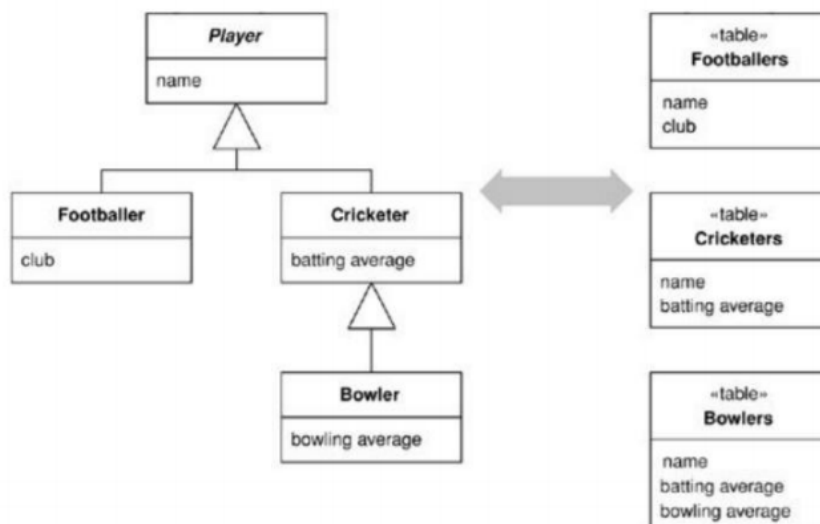
SINGLE TABLE INHERITANCE



74

Metoda 2:

CONCRETE TABLE INHERITANCE



75

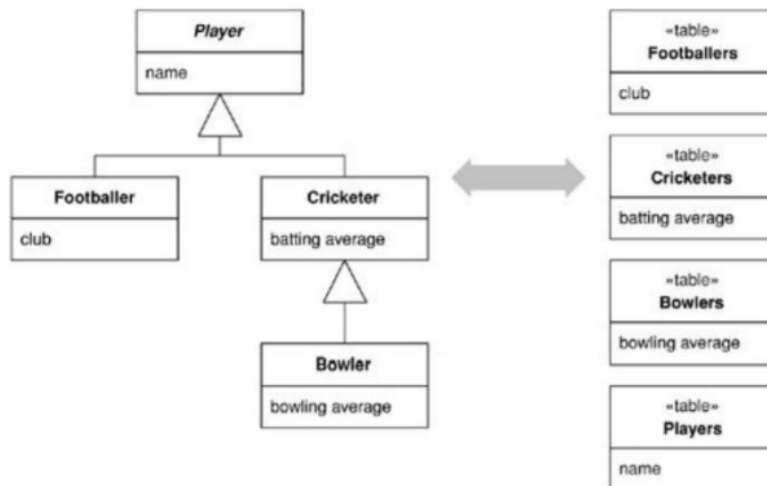
<<table>> Customer
name
preferences

<<table>> Employee
name
salary

<<table>> Executive
name
salary
bonus

Metoda 3:

CLASS TABLE INHERITANCE



THEORY

Software Design Techniques

- Set of practices for analysing, decomposing, and modularising software system architectures.
- Telling it how it should work, rather than what it should do

Stiva de tehnologii $\neq$ architectura

When to have **static**?

When to have **final**?

Overloading methods : Same name, different params

Overriding methods : Same name, same params, different functionality

- Final methods cannot be overridden!
- Final classes cannot be subclassed

Composition

- Reuse objects in order to create more complex objects
- Represents a HAS-A relationship

Polymorphism

- The possibility of an instance (object) to have different types

SOLID : class design principles

S > Single Responsibility (solve with inheritance)

- Should only have one reason to change, be responsible for one user, actor

O > Open-Closed (solve with interface or polymorphism)

- Software artifacts should be open for extension but closed for modification
- Compozitia este mai flexibila decat mostenirea

L > Liskov Substitution Principle

- Fiecare obiect al unei superclase poate fi inlocuit de orice obiect al unei subclase

I > Interface Segregation

- Clients should not be forced to depend upon interfaces that they don't use

D > Dependency Inversion

- High-level modules should not depend on low-level modules
- Abstractions should not depend on details, but viceversa

Design by contract

- Precondition : Subclass methods are no stronger than the base class
- Postcondition: Subclass methods are no weaker than the base class

How not to SOLID?

- Single responsibility -> thousands of very small classes
- Open-closed -> inheritance for every new feature
- Interface segregation -> everything is single-inheritance based
- Dependency inversion -> everything is abstracted (data-driven to extreme)

GRASP : General Responsibility Assignment Software Patterns

- Objects communicating with each other to complete operations
- Issues: responsibility assigned to obj, interaction ways between obj

Knowing responsibilities (grasp)

- Knowing about private encapsulated data
- Knowing about related objects
- Knowing about things it can derive or calculate

Doing responsibilities (grasp)

- Doing something itself
- Initiating action in other objects
- Controlling and coordinating activities in other objects

GRASP principles

- Expert
- Creator
- Controller
- Low Coupling
- High Cohesion
- Polymorphism
- Pure Fabrication
- Indirection
- Don't Talk to Strangers (Law of Demeter)

Low Coupling : how strongly are the objects connected to each other?

- Coupling = objects depending on other objects

High Cohesion : how are the operation

- Related responsibilities are placed into one manageable unit
- Benefits: easily understandable and maintainable, code reuse, low coupling

GRASP : information expert

- We should assign a responsibility to the class that has the necessary information to fulfil the responsibility

GRASP : benefits

- Information encapsulation is maintained since objects use their own information to fulfil tasks -> supports low coupling
- Behavior is distributed across the classes that have the required information
- > more cohesive “lightweight” class definitions

Creator (who should create a new instance of some class?)

- Guides assigning responsibilities related to the creation of objects
- Benefit: Supports low coupling
- Issues: complex creation procedures

Controller (who should be responsible for handling a system event?)

- Controller is a non-user interface obj responsible for handling a sys event
- Facade controller; Role controller; Use-case controller
- Controller delegates the work to the objects, it coordinates the activity
- Facade controller -> when there are not “too many” system events
- Use-case controller -> if facade would lead to low cohesion or high coupling
- Controller is **bloated** if: too many responsibilities or performs many task itself
- Benefits: increase potential for reusable components; reason about use cases

Law of Demeter

- You can only use things that are allowed for you
- Benefits: coupling control, information hiding, information restriction, few interfaces, explicit interfaces

Higher-Level Design

- Packages : logical grouping of declarations that can be imported
- Goal : **partition** the classes based on a **criteria** and then **allocate** to packages
- Issues : best partitioning criteria, what principles govern the design

Cohesion Principles (package design)

- Reuse/Release equivalence principle (REP)
- Common Reuse Principle (CRP)
- Common Closure Principle (CCP)

Coupling Principles

- Acyclic Dependencies Principle (ADP)
- Stable Dependencies Principle (SDP)
- Stable Abstractions Principle (SAP)

What is really reusability?

- Reuse code if, and only if, I never need to look at the source-code
- Reused code like product -> don't have to maintain it

Reuse/Release Equivalency Principle (REP)

- Only what is released can be efficiently reused
- Either all the classes in a package are reusable or none of it is!

Common Reuse Principle (CRP)

- Classes in a package should be reused together
- If I depend on a package, I want to depend on every class in that package!
- We groupe classes in a package if we would reuse them together

Common Closure Principle (CCP)

- The classes in a package should be closed against the same kind of changes
- A change that affects a package affects all the classes in it
- We groupe classes based on maintainability, classes that tend to change together for the same reason

Rep and CRP makes life easier for reuser (small packages)

CCP makes life easier for maintainer (larger packages)

Early we use CCP, then if it stabilizes we switch for REP and CRP.

Acyclic Dependencies Principle (ADP)

- Dependency structure for the released component must be a Directed Acyclic Graph without cycles.

Offline Concurrency Pattern

Multiple threads to manipulate same data -> Transaction managers

ACID (atomic -> consistent -> isolated -> durable)

Concurrency Problems

- Lost updates
- Inconsistent reads => correctness failure
- Liveness? -> how much concurrency can the system handle?

Request : single call from outside which gets processed, optionally has a response

Session : a long running interaction between a client and a server

Process : execution context that provides a lot of isolation for the internal data

Thread : active agent that's set up so that multiple threads can operate in 1 process

Isolation : partition the data so that any of it can only be accessed by 1 active agent

Immutable data : separate data that cannot be modified

Mutable data that cannot be isolated -> **concurrency control**

PG. 508 - 514: lots of crazy diagrams

Architectural Patterns

Definition

“An architectural pattern expresses a fundamental structural organization schema for software systems. It provides a set of predefined subsystems, specifies their responsibilities, and includes rules and guidelines for organizing the relationships between them.”

A clean architecture is:

- Independent of frameworks
- Testable : Should be able to test without the UI
- Independent of the UI, of the database, of any external agency

Architectural Pattern vs Design Pattern

Construction analogy: Architectural Patterns refer to how the application is structured (it tells us if the building is a house, office, commercial centre), while Design Patterns refer to how the components are built (how each room or floor looks like in more detail).

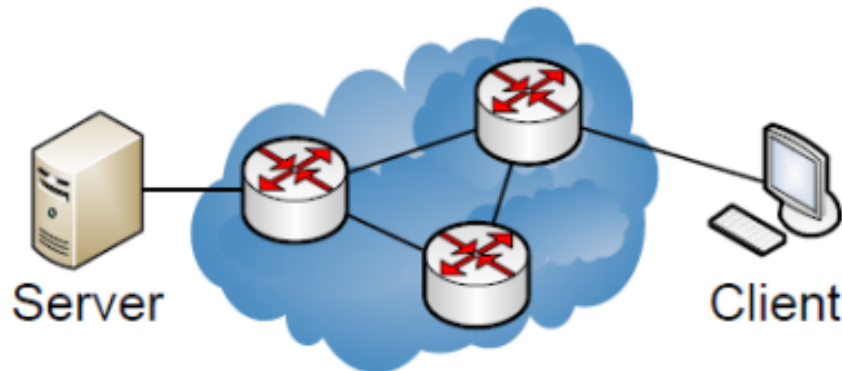
Layered

For applications that can be decomposed into groups (Layers), where each layer has a different level of abstraction. A higher level layer depends on the lower level layers (for the data to get to the Presentation Layer, it first has to be processed in the Business Layer, which in turn has to be retrieved using the Data Access Layer).

- 3-layered architecture:
 - > Presentation layer (== UI)
 - > Business logic layer (a.k.a domain layer, data processing)
 - > Data access layer (== database access)

Client-server

- Consists of one server, and multiple clients
- The server will provide services, and clients can request services
- Good for: online applications such as email, document sharing etc.



Master-Slave

- The master component distributes the work among identical slave components and computes a final result from the results which the slaves return
- Good when: peripherals are connected to a bus in a computer system
-

Peer-to-Peer (P2P)

- Individual components known as peers
- Peers may function both as a client, requesting services from other peers, and as a server, providing services to other peers.
- A peer may act as a client or as a server or as both, and it can change its role dynamically
- Usage: crypto, blockchain

Model View Controller (MVC)

- Constraints: the display and behavior of the application must reflect data manipulations immediately; easy to change UI

Model

- Encapsulates and manipulates the domain data to be rendered
- Has no idea how to display the information nor does it interact with the user
- Has functionality to manipulate, obtain, and deliver data to others

View

- Specific visual rendering of the information contained in the model
- Each view is dependent of a model

Controller

- Handles user input and handle requests using the model and views
- Allows the decoupling of the model and its views, allowing views to simply render data and models to simply encapsulate data
- The behavior of the controller is dependent upon the state of the model

MVC Benefits

- Multiple views of the same model, views can be synchronized
- “Pluggable” views and controllers
- Exchangeability of “look and feel”
- Framework potential

MVC liabilities

- Increased complexity
- Potentially excessive number of updates
- Intimate connection between view and controller
- Close coupling of views and controllers to a model
- Inefficiency of data access in view
- Inevitability of change to view and controller when porting

Event-driven Architectures

Used for projects that need to be highly scalable, adaptable and the components highly decoupled. It is an asynchronous pattern. In everyday words, this architectural pattern works for applications where usually nothing happens, but an event (like a reading from a smoke sensor) needs to be dealt with accordingly.

Broker

- Used to structure distributed software systems
- A broker component is responsible for coordinating communication
- Constraints : components should be able to access services provided by others; architecture should hide system- and implementation-specific details
- Non-Distributed system -> use together with proxy
- Benefits : location transparency, portability of broker system, interoperability between broker systems, reusability
- Liabilities : reliability, lack of atomic transactions

Mediator

- For events that have multiple steps and require some level of orchestration to process the events
- Event queue -> event mediator -> event channel -> event processor
- Interactive systems

Service Oriented Architectures (SOA)

Business functionalities - aka. Services (like Check Inventory, Check Order Status) - are designed to be reusable, with low coupling, independent of each other (but a Service can be composed of more services) and to abstract the logic behind (a developer can use the service to Check Inventory, but doesn't need to know how that is done exactly).

Design Principles

- Services are reusable
- Services share a **formal contract** (this is how they communicate)
- Services are **loosely coupled**
- Services **abstract** underlying logic
- Services are composable
- Services are **autonomous**
- Services are stateless
- Services are discoverable
- Services have a network-addressable interface
- Services are location transparent

REST (REpresentation State Transfer)

- Idea: Self-contained requests specify what resource to operate on and what to do to it
- Principles
- The key abstraction of information is a **resource**
- The **representation** of a resource is a sequence of bytes + metadata
- All interactions are context-free (**stateless**)
- Components perform only a small set of well-defined methods (uniform interface)
- Supports caching and representation reuse
- The presence of intermediaries is promoted (Links between resources)

SOAP Services are like envelopes while REST services are like postcards (security)

SOAP Services cannot use REST while REST can use SOAP

SOAP vs REST

SOAP VS. REST

Difference	SOAP	REST
Style	Protocol	Architectural Style
Function	Function-driven: transfer structured information	Data-driven: access a resource for data
Data format	Only uses XML	Permits several data formats: HTML, XML, plain text, JSON
Security	Supports WS-Security and SSL	Supports SSL and HTTPS
Bandwidth	Requires more resources and bandwidth	Requires fewer resources and is lightweight
Data cache	Cannot be cached	Can be cached
Payload handling	Has a strict communication contract and needs knowledge of everything before any interaction	Needs no knowledge of the API
Transactional support	Has built-in ACID compliance	Lacks ACID compliance

Microservices Architecture

SOA vs Microservices: SOA in enterprise level, used in huge applications, where an application is composed of more, smaller applications. Microservices is application level. It enables the internals of a single application to be broken up into small pieces that can be independently changed, scaled, and administered. It does not define how applications talk to one another—for that we are back to the enterprise scope of the service interfaces provided by SOA.

SOA	VS.	MICROSERVICES
▪Share-as-much-as-possible ▪Uses orchestration and choreography ▪Uses Message middleware ▪Coarse-grained services ▪No pre-described limits as to which remote-access protocols can be used		▪Share-as-little-as-possible ▪Favorizes choreography ▪Uses API layer as service access façade ▪Fine-grained services ▪rely on 2 different remote-access protocols to access services: REST and simple messaging (JMS, MSMQ, AMQP, etc.)

iDesign Architecture

Never build an architecture that satisfies requirements perfectly - NO functional decomposition. Requirements of an application will always change (because they simply do, needing more functionality or maybe one is not used as much, so it is to be removed). The answer: Volatility driven decomposition (volatility = liable to change rapidly and unpredictably, especially for the worse): Identify core functionalities (what the app will always have to do), Identify volatilities (what is likely to change and what we need to prepare for), Define services that will compose the architecture. Generally an app can be structured into 4 parts:

Client - Who is using the app or making a request

Manager - What needs to be done

Engine - How things are implemented

Resource Access - Where is the data obtained from

Design patterns

- Creational: deal with creating classes and objects (e.g. builder, factory, prototype etc.)
- Structural: deal with decoupling the interface and implementation of classes and objects (e.g. adapter, bridge, decorator, proxy etc.)
- Behavioral: deal with dynamic interactions among societies of classes and objects (command, observer etc.)

Drawbacks:

- do not lead to direct code reuse
- are deceptively simple
- teams may suffer from patterns overload
- needs work from humans

Creational DP

- System independent how the objects are created
- Focus on defining small behaviours and combine to create complex ones
- Advantages:
 - flexibility
 - configure system with product objects that can vary
 - create standard or uniform
 - configuration can be static or dynamic
 - **!! FAVOR COMPOSITION OVER INHERITANCE !!**
 - Factory Methods or Prototypes can handle each type of product but it gets hard to insure the wrong types of items are not used together

Factory method

- define interface to create an object, but let subclasses decide which class to instantiate
- a.k.a. Virtual Constructor
- applicability:
 - A class can't anticipate the class of objects it must create
 - A class wants its subclasses to specify the objects it creates
 - Classes delegate responsibility to one of several helper subclasses

PARTICIPANTS & COLLABORATIONS

Product

- defines the interface of objects that will be created by the FM
- Concrete Product implements the interface

Creator

- declares the FM, which returns a product of type Product.
- may define a default implementation of the FM

ConcreteCreator

- overrides FM to provide an instance of ConcreteProduct

- consequences: subclasses can change the product that is created

Prototype pattern

- Specify the kinds of objects to create using a prototypical instance
- Create new objects by copying this prototype
- We use it when:
 - > client class should be independent of how its products are created, composed, and represented
 - > the classes to instantiate are specified at run-time

PARTICIPANTS & COLLABORATIONS

Prototype

- declares an interface for cloning itself.

ConcretePrototype

- implements an operation for cloning itself.

Client

- creates a new object by asking a prototype to clone itself.

- Consequences:
 - > add and remove products at run-time

> Each subclass of Prototype must implement clone: difficult when classes already exist

- Implementation issues:

> number of prototype is not fixed -> keep a registry -> prototype manager

> must write an Initialize method

Abstract factory

- Provides an interface for creating families of related or dependent objects without specifying their concrete classes

- System should be independent of how its products are created, composed and represented

- Need to enforce that a family of product objects is used together

PARTICIPANTS & COLLABORATIONS

Abstract Factory

- declares an interface for operations to create abstract products

ConcreteFactory

- implements the operations to create products

AbstractProduct

- declares an interface for a type of product objects

ConcreteProduct

- declares an interface for a type of product objects

Client

- uses only interfaces decl. by AbstractFactory and AbstractProduct

- Consequences:

> isolation of concrete classes

> exchanging of product families becomes easy

> promotes consistency among products

> Supporting new kinds of products is difficult requires a change in the interface of AbstractFactory ... and consequently all subclasses

Singleton pattern

- Ensure a class has only one instance and provide a global point of access to it
- accessible to clients from one point
- want the instance to be extensible
- can also allow a countable number of instances

PARTICIPANTS AND COLLABORATIONS

Singleton

- defines an **Instance** method that becomes the single "gate" by which clients can access its unique instance.
 - **Instance** is a class method (i.e. static)
- may be responsible for creating its own unique instance

- Consequences:
 - > controlled access to sole instance
 - > reduced global name space
 - > permits refinement of operations and representation
- Some say that it is an Anti-pattern

Structural DP

- concerned with how classes and objects are composed to form larger structures
- Structural class patterns use inheritance to compose interfaces or implementations

- Structural objects describe ways to compose objects to realize new functionality

Adapter pattern

- Convert the interface of a class into another interface clients expect
- Enables classes to work together that couldn't otherwise because of incompatible interfaces
- A.k.a. Wrapper
- Create a reusable class that cooperates with unrelated or unforeseen classes
- Inheritance based or Composition based

ADAPTER PATTERN - PARTICIPANTS

Target

- defines the domain-specific interface that Client uses.

Client

- collaborates with objects conforming to the Target interface.

Adaptee

- defines an existing interface that needs adapting.

Adapter

- adapts the interface of Adaptee to the Target interface.

Composite pattern

- Nightmare Implementation (a.k.a BAD):
 - > for each operation deal with each category of objects individually
 - > no uniformity and no hiding of complexity

> a lot of code duplication

- Treat individual objects and compositions of these object uniformly
- Be able to ignore the difference between compositions of objects and individual objects

PARTICIPANTS & COLLABORATIONS

Component

- declares interface for objects in the composition
- implements default behavior for components when possible

Composite

- defines behavior for components having children
- stores child components
 - implement child-specific operations

Leaf

- defines behavior for primitive objects in the composition

Client

- manipulates objects in the composition through the Component interface

- Consequences (a.k.a GOOD):
 - > Defines uniform class hierarchies
 - > It avoids to deal in a different manner with each class => simpler code
 - > Easier to extend
- Placing the container operations:
 - > Pro-Transparency == place in Component
 - > Pro-Safety == place in Composite

Decorator pattern

- Strategies: Object Composition or “Change the skin, not the guts”
- Add responsibilities to a particular object rather than its class
- Attach additional responsibilities to an object (i.e. without affecting other objects)

PARTICIPANTS & COLLABORATIONS

Component

- defines the interface for objects that can have responsibilities added dynamically

ConcreteComponent

- the "bases" object to which additional responsibilities can be added

Decorator

- defines an interface conformant to Component's interface
 - for transparency
- maintains a reference to a Component object

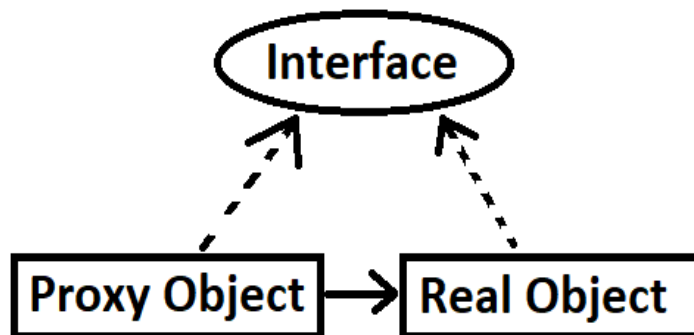
ConcreteDecorator

- adds responsibilities to the component

- Consequences:
 - > more flexibility
 - > “pay-as-you-go” approach
 - > easy to customize, but hard to learn and debug
- Issues:
 - > don't put data members in Component (== keep decorators lightweight)
- Changing the “guts” of an object:
 - > "shield" the implementation from direct access (Proxy)

Proxy pattern

- placeholder for the real object:



> both objects implement the same interface, but the first we call is the Proxy Object, and if needed, we call the real one too. For ex: in proxy we have some security checks, if passed, call the real object

> Usage: photos on hard drive and showing them: we have a lot of images, but if load them all it will quickly eat all the memory. We use a proxy object that will load only those images that are „visible on the screen,, (using flags to check if visible)

PARTICIPANTS

Proxy

- maintains a reference that lets the proxy access the real subject.
- provides an interface identical to Subject's
 - so that proxy can be substituted for the real subject
- controls access to the real subject
 - may be responsible for creating or deleting it

Subject

- defines the common interface for RealSubject and Proxy

RealSubject

- defines the real object that the proxy holds place for

- Consequences:

> introduces a level of indirection

Bridge pattern

- decouple an abstraction from its implementation
- allow implementation to vary independently from its abstraction
- an abstraction can use different implementations
- an implementation can be used in different abstractions
- avoid permanent binding btw. an abstraction and its implementation
- hide the implementation of an abstraction completely from clients

PARTICIPANTS

Abstraction

- defines the abstraction's interface
- maintains a reference to an object of type Implementor

Implementor

- defines the interface for implementation classes
- does not necessarily correspond to the Abstraction's interface
- Implementor contains primitive operations,
- Abstraction defines the higher-level operations based on these primitives

RefinedAbstraction

- extends the interface defines by Abstraction

ConcreteImplementer

- implements the Implementor interface, defining a concrete impl.

- Consequences:

> Decoupling interface and implementation

> Implementation is configurable and changeable at run-time

Adapter vs Bridge

- Common:
 - > promote flexibility by providing a level of indirection to another object
- Differences:
 - > Adapter resolves incompatibilities between two existing interfaces. No focus on how those interfaces are implemented
 - > Bridge links an abstraction and its implementations. It provides a stable interface
- **The Adapter pattern makes things work after they're designed; Bridge makes them work before they are.**

Behavioural DP

- concerned with algorithms and the assignment of responsibilities between objects
- describe patterns of communication between classes/objects

Observer pattern

E.g. Weather monitoring app

- By coding to concrete implementations there is no way to add additional display elements without making code change
- Defines a one-to-many dependency between objects so that when one object changes state
- Notify when an object changes its state. To avoid spamming „clients,, that are not interested, we have the opportunity to “unsubscribe from the newsfeed“
- Power of loose coupling:
 - > observers can be added at any time
 - > subjects and observers can be reused or modified
- > the only thing subject knows about an observer is that it implements an interface

Strategy pattern

- Define a family of algorithms, encapsulate each one, and make them interchangeable
- Good when:
 - > You need different variants of an algorithm
 - > Many related classes differ only in their behavior

PARTICIPANTS

Strategy

- declares an interface common to all supported algorithms.
- Context uses this interface to call the algorithm defined by a ConcreteStrategy

ConcreteStrategy

- implements the algorithm using the Strategy interface

Context

- configured with a ConcreteStrategy object
- may define an interface that lets Strategy objects to access its data

- Consequences: > families of related algorithms usually provide different implementations of the same behavior

State pattern

- SPOP = Simple Post Office Protocol
- Used to download emails from server
- Allow an object to alter its behavior when its internal state changes
- Object will appear to change its class
- Good when:

- > object's behavior depends on its state
- > it must change behavior at run-time depending on that state

State vs Strategy

Rate of Change

- Strategy
 - Context object usually contains one of several possible **ConcreteStrategy** objects
- State
 - Context object often changes its **ConcreteState** object over its lifetime

Visibility of Change

- Strategy
 - All **ConcreteStrategy** do the same thing, but differently
 - Clients do not see any difference in behavior in the Context
- State
 - **ConcreteState** acts differently
 - Clients see different behavior in the Context

47

Blackboard pattern

- Useful for problems with no deterministic strategies
- Components:
 - > blackboard: global memory containing objects
 - > knowledge source: specialized modules with their own representation
 - > control component: selects, configures and executes modules
- All the components have access to the blackboard
- Usage: speech recognition, sonar signal interpretation etc.

Command pattern

- We create instances of command objects and save in these all the requested commands and we (can) put them in a queue (se poate da undo/CTRL+Z, se poate vedea istoricul etc).
- A.k.a. Encapsulate requests as objects
- Because it supports the “undo” -> recover from crashes
- Consequences:
 - > Decouples Invoker from Receiver (can be extended)
 - > easy to add new commands (it is open-closed)

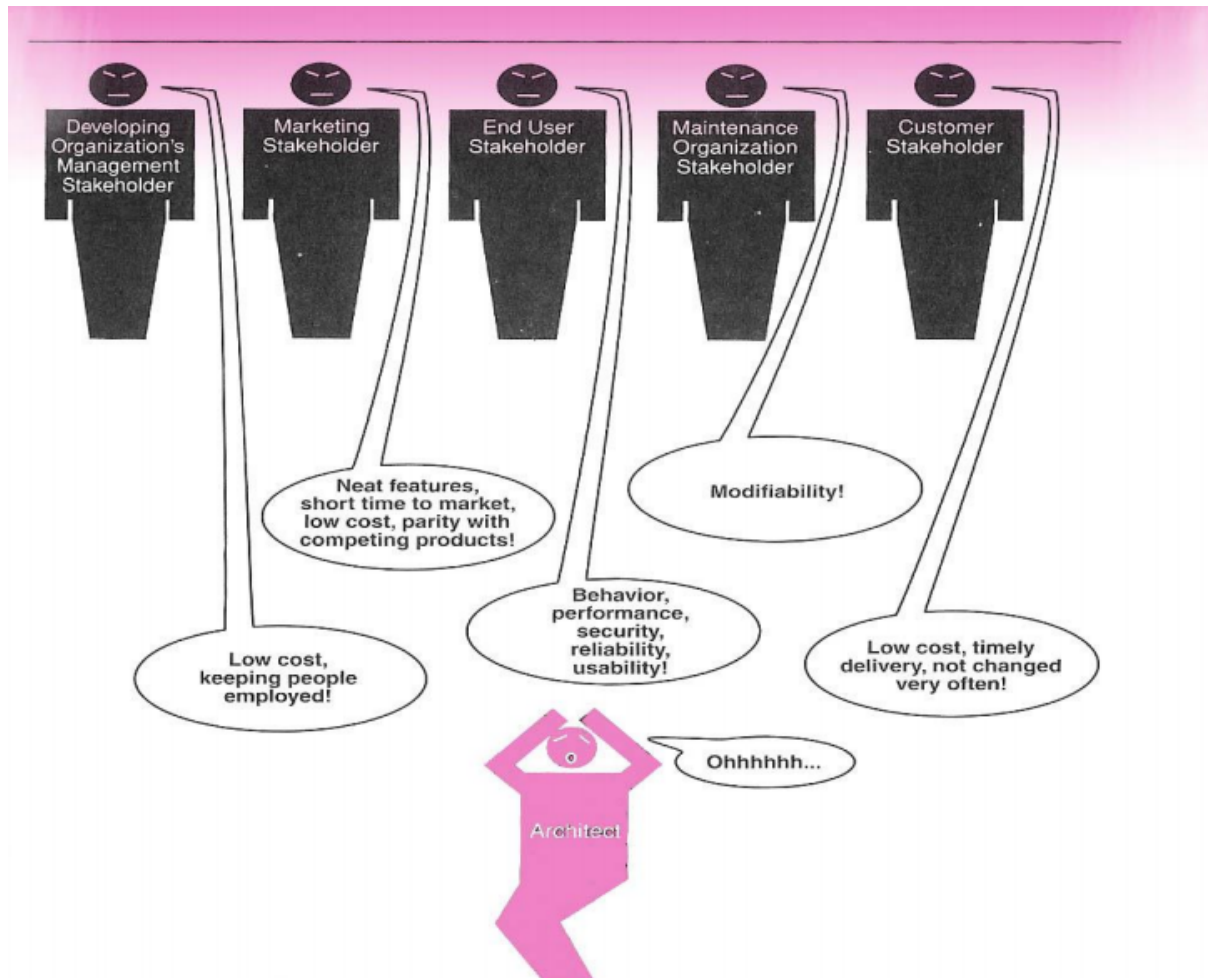
Chain of responsibility pattern

- Decouple sender of request from its receiver
- Put receivers in a chain and pass the request along the chain, until an object handles it
- Use when:
 - > more than one object may handle a request
 - > send a request to several objects without specifying the receiver
- Consequences:
 - > Reduced Coupling <-> frees the client (sender) from knowing who will handle its request
 - > Flexibility <-> chain can be modified at run-time

Decorator vs Chain of responsibility

Chain of Responsibility	Decorator
Comparable to "event-oriented" architecture	Comparable to layered architecture (layers of an onion)
The "filter" objects are of equal rank	A "core" object is assumed, all "layer" objects are optional
User views the chain as a "launch and leave" pipeline	User views the decorated object as an enhanced object
A request is routinely forwarded until a single filter object handles it. many (or all) filter objects <i>could</i> contrib. to each request's handling.	A layer object always performs pre or post processing as the request is delegated.
All the handlers are peers (like nodes in a linked list) – "end of list" condition handling is required.	All the layer objects ultimately delegate to a single core object - "end of list" condition handling is not required. ⁷⁰

Quality attributes/requirements



- Requirements elicitation = the activities involved in discovering the requirements of the system
- Techniques when thinking on the problem: asking, analysis, prototyping etc.
- FURPS:

Functionality

- feature sets
- capabilities
- security

Usability

- human factors
- aesthetics
- consistency in the user interface
- online and context-sensitive help
- wizards and agents
- user documentation

Reliability

- frequency and severity of failure
- recoverability
- predictability
- accuracy

Performance

- speed
- efficiency
- availability
- accuracy
- response time
- recovery time
- resource usage

Supportability

- testability
- extensibility
- adaptability
- maintainability
- compatibility
- configurability
- installability
- localizability (internationalization)

- Best practices in order to create the best project ever 🌈 (more stuff about this in Lecture 12)

